

module-06-strings

Chapter 6: Manipulating Strings | 46 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). Syllabus Ch6: literals, indexing/slicing, in/not in, case tests, join/split, justify, strip, pyperclip projects

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

Strings are text lists of characters, so indexing/slicing/in work like lists. The power tools: upper()/lower() for case, isX() tests (isalpha/isdigit/isspace...), startswith()/endswith(), join() to glue with a separator, split() to break apart, rjust()/ljust()/center() to align, strip()/rstrip()/lstrip() to trim whitespace. Escape with backslash (\n newline, \t tab) or raw strings r'...'. pyperclip lets programs talk to the clipboard.

Key ideas to carry forward

- spam[0], spam[1:5] work on strings exactly like lists.
- ','.join(['a','b']) -> 'a,b'; 'a,b'.split(',') -> ['a','b'].
- 'Hello'.upper().isupper() chains: convert then test.
- Password-locker, bullet-adder, and table-printer are the three capstone automations.

Classic beginner mistakes

- ! Forgetting raw strings for backslashes ('C:\\new' vs r'C:\new').
- ! upper() returns a NEW string; spam stays unchanged unless reassigned.

Study order hint: 01-08 literals/escapes, 09-20 indexing + case/isX, 21-34 join/split/justify/strip/clipboard, 35-46 the three projects.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. How do indexing/slicing transfer from lists to strings?

A: Identically: `s[0]`, `s[-1]`, `s[1:4]`. Difference: strings are immutable - no item assignment.

Q2. join vs split?

A: `sep.join(list)` glues items with `sep`; `s.split(sep)` breaks into a list. They are inverses.

Q3. What do the `isX` methods test?

A: `isalpha/isdigit/isalnum/isspace/islower/isupper` classify the whole string - ideal for validating input like ages.

Q4. `startswith/endswith` use cases?

A: File filtering (`'report'.endswith('.pdf')`), URL routing, quick type checks without regex.

Q5. `rjust/ljust/center` and `strip`?

A: Padding for tables/receipts; `strip` variants trim whitespace or given chars from the edges.

Q6. Raw strings and escapes?

A: `r'C:\new'` keeps backslashes literal; otherwise `\n`, `\t`, `\\`, `\'` are control/quote escapes.

Teach it to others

The one analogy that sticks

Strings are bead necklaces: each bead has a position (index), you can restring sections (slices) but never repaint a bead in place (immutable) - you build a new necklace (methods return new strings).

What your students will get wrong

- x `upper()` changes the variable (returns new; reassign).
- x `split()` defaults are obvious (splits on any whitespace runs).
- x Backslashes are literal in normal strings (they escape).

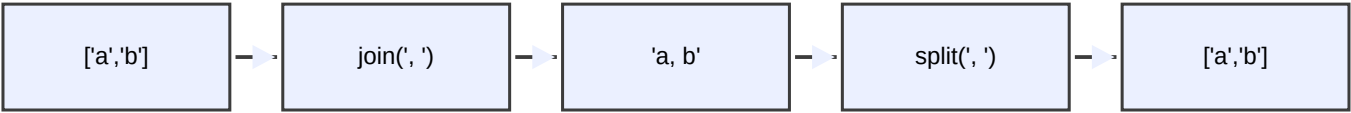
5-minute whiteboard plan

1. Classify 'Hello123' with `isX` chain live.
2. Build a receipt line with `ljust/rjust`.
3. Students fix a path bug with raw strings.
4. Exit ticket: join a bullet list with `'\n* '`.

Visualize it - draw these from memory

Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - join and split are inverses



Glue with join, break with split. Round-trip them to prove they undo each other.

Visual 2 - methods return NEW necklaces

0	1	2	3	4
H	e	l	l	o

s[0]='H', s[1:4]='ell'. upper() builds a new necklace; the old one never changes unless reassigned.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Email validator (no regex)

```
e = 'al@example.com'
print('@' in e and '.' in e and not e.startswith('@'))
print(e.split('@'))
# EXPECTED: True / ['al', 'example.com']
```

X2. Initials maker

```
name = 'ada lovelace'
parts = name.split()
print((parts[0][0] + parts[1][0]).upper())
# EXPECTED: AL
```

X3. Receipt line

```
item, price = 'Apples', 2.5
print(item.ljust(12, '.') + str(price).rjust(6))
# EXPECTED: Apples..... 2.5
```

Program map - all 46 at a glance

```
01 01_string_literals.py
02 02_string_literals_spam.py
03 03_string_literals_hello.py
04 04_string_literals_2.py
05 05_string_literals_dear.py
06 06_string_literals_dear_2.py
07 07_string_literals_dear_3.py
08 08_string_literals_written.py
09 09_indexing_slicing_strings.py
10 10_indexing_slicing_strings_spam.py
11 11_indexing_slicing_strings_spam_2.py
12 12_not_operators_strings.py
13 13_upper_lower_isupper.py
14 14_upper_lower_isupper_how.py
15 15_upper_lower_isupper_how_2.py
16 16_upper_lower_isupper_spam.py
17 17_upper_lower_isupper_hello.py
18 18_isx_string_methods.py
19 19_isx_string_methods_while.py
20 20_isx_string_methods_enter.py
21 21_startswith_endswith_string.py
22 22_join_split_string.py
23 23_join_split_string_my.py
24 24_join_split_string_myabcnameabc.py
25 25_join_split_string_spam.py
26 26_justifying_text_rjust.py
27 27_justifying_text_rjust_hello.py
28 28_justifying_text_rjust_hello_2.py
29 29_justifying_text_rjust_printpicnic.py
30 30_justifying_text_rjust_picnic.py
31 31_removing_whitespace_strip.py
32 32_removing_whitespace_strip_spam.py
33 33_copying_pasting_strings.py
34 34_copying_pasting_strings_pyperclip.py
35 35_step_1_program.py
36 36_step_2_handle.py
37 37_step_3_copy.py
38 38_step_3_copy_2.py
39 39_project_adding_bullets.py
```

40 40_project_adding_bullets_lists.py
41 41_step_1_copy.py
42 42_step_2_separate.py
43 43_step_2_separate_pyperclip.py
44 44_step_3_join.py
45 45_table_printer.py
46 46_table_printer_apples.py

Topic: String Literals

01/46 01_string_literals.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = "That is Alice's cat."
```

02/46 02_string_literals_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
spam = 'Say hi to Bob\'s mother.'
```

03/46 03_string_literals_hello.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
print("Hello there!\nHow are you?\nI\'m doing fine.")
```

```
# OUT: Hello there!
```

```
# OUT: How are you?
```

```
# OUT: I\'m doing fine.
```

04/46 04_string_literals_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
print(r'That is Carol\'s cat.')  
# OUT: That is Carol\'s cat.
```

05/46 05_string_literals_dear.py [script]

Script: save and run; predict the output first, then compare.

```
print('''Dear Alice,  
  
Eve's cat has been arrested for catnapping, cat burglary, and extortion.  
  
Sincerely,  
Bob''')
```

06/46 06_string_literals_dear_2.py [script]

Script: save and run; predict the output first, then compare.

Dear Alice,

Eve's cat has been arrested for catnapping, cat burglary, and extortion.

Sincerely,

Bob

07/46 07_string_literals_dear_3.py [script]

Script: save and run; predict the output first, then compare.

```
print('Dear Alice,\n\nEve\'s cat has been arrested for catnapping, cat  
burglary, and extortion.\n\nSincerely,\nBob')
```

08/46 08_string_literals_written.py [script]

Script: save and run; predict the output first, then compare.

```
"""This is a test Python program.
Written by Al Sweigart al@inventwithpython.com

This program was designed for Python 3, not Python 2.
"""

def spam():
    """This is a multiline comment to help
    explain what the spam() function does."""
    print('Hello!')
```

Topic: Indexing and Slicing Strings

09/46 09_indexing_slicing_strings.py [script]

Script: save and run; predict the output first, then compare.

'	H	e	l	l	o		w	o	r	l	d	!	'
0	1	2	3	4	5		6	7	8	9	10	11	

10/46 10_indexing_slicing_strings_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = 'Hello world!'  
spam[0]  
# OUT: 'H'  
spam[4]  
# OUT: 'o'  
spam[-1]  
# OUT: 'l'  
spam[0:5]  
# OUT: 'Hello'  
spam[:5]  
# OUT: 'Hello'  
spam[6:]  
# OUT: 'world!'
```

11/46 11_indexing_slicing_strings_spam_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = 'Hello world!'
```

```
fizz = spam[0:5]
```

```
fizz
```

```
# OUT: 'Hello'
```

Topic: The in and not in Operators with Strings

12/46 12_not_operators_strings.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
'Hello' in 'Hello World'
```

```
# OUT: True
```

```
'Hello' in 'Hello'
```

```
# OUT: True
```

```
'HELLO' in 'Hello World'
```

```
# OUT: False
```

```
' ' in 'spam'
```

```
# OUT: True
```

```
'cats' not in 'cats and dogs'
```

```
# OUT: False
```

Topic: The upper(), lower(), isupper(), and islower() String Methods

13/46 13_upper_lower_isupper.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = 'Hello world!'
```

```
spam = spam.upper()
```

```
spam
```

```
# OUT: 'HELLO WORLD!'
```

```
spam = spam.lower()
```

```
spam
```

```
# OUT: 'hello world!'
```

14/46 14_upper_lower_isupper_how.py [script]

Script: save and run; predict the output first, then compare.

```
print('How are you?')
feeling = input()
if feeling.lower() == 'great':
    print('I feel great too.')
else:
    print('I hope the rest of your day is good.')
```

15/46 15_upper_lower_isupper_how_2.py [script]

Script: save and run; predict the output first, then compare.

```
How are you?
```

```
GREat
```

```
I feel great too.
```

16/46 16_upper_lower_isupper_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = 'Hello world!'  
spam.islower()  
# OUT: False  
spam.isupper()  
# OUT: False  
'HELLO'.isupper()  
# OUT: True  
'abc12345'.islower()  
# OUT: True  
'12345'.islower()  
# OUT: False  
'12345'.isupper()  
# OUT: False
```

17/46 17_upper_lower_isupper_hello.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
'Hello'.upper()  
# OUT: 'HELLO'  
'Hello'.upper().lower()  
# OUT: 'hello'  
'Hello'.upper().lower().upper()  
# OUT: 'HELLO'  
'HELLO'.lower()  
# OUT: 'hello'  
'HELLO'.lower().islower()  
# OUT: True
```

Topic: The isX String Methods

18/46 18_isx_string_methods.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

'hello'.isalpha()
# OUT: True
'hello123'.isalpha()
# OUT: False
'hello123'.isalnum()
# OUT: True
'hello'.isalnum()
# OUT: True
'123'.isdecimal()
# OUT: True
'   '.isspace()
# OUT: True
'This Is Title Case'.istitle()
# OUT: True
'This Is Title Case 123'.istitle()
# OUT: True
'This Is not Title Case'.istitle()
# OUT: False
'This Is NOT Title Case Either'.istitle()
# OUT: False
```

19/46 19_isx_string_methods_while.py [script]

Script: save and run; predict the output first, then compare.

```
while True:
    print('Enter your age:')
    age = input()
    if age.isdecimal():
        break
    print('Please enter a number for your age.')

while True:
    print('Select a new password (letters and numbers only):')
    password = input()
    if password.isalnum():
        break
    print('Passwords can only have letters and numbers.')
```

20/46 20_isx_string_methods_enter.py [script]

Script: save and run; predict the output first, then compare.

```
Enter your age:
forty two
Please enter a number for your age.
Enter your age:
42
Select a new password (letters and numbers only):
secr3t!
Passwords can only have letters and numbers.
Select a new password (letters and numbers only):
secr3t
```

Topic: The startswith() and endswith() String Methods

21/46 21_startswith_endswith_string.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
'Hello world!'.startswith('Hello')  
# OUT: True  
'Hello world!'.endswith('world!')  
# OUT: True  
'abc123'.startswith('abcdef')  
# OUT: False  
'abc123'.endswith('12')  
# OUT: False  
'Hello world!'.startswith('Hello world!')  
# OUT: True  
'Hello world!'.endswith('Hello world!')  
# OUT: True
```

Topic: The join() and split() String Methods

22/46 22_join_split_string.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> ','.join(['cats', 'rats', 'bats'])  
# OUT: 'cats, rats, bats'  
  
>>> ' '.join(['My', 'name', 'is', 'Simon'])  
# OUT: 'My name is Simon'  
  
>>> 'ABC'.join(['My', 'name', 'is', 'Simon'])  
# OUT: 'MyABCnameABCisABCSimon'
```

23/46 23_join_split_string_my.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
'My name is Simon'.split()  
# OUT: ['My', 'name', 'is', 'Simon']
```

24/46 24_join_split_string_myabcnameabc.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
'MyABCnameABCisABCSimon'.split('ABC')  
# OUT: ['My', 'name', 'is', 'Simon']  
'My name is Simon'.split('m')  
# OUT: ['My na', 'e is Si', 'on']
```

25/46 25_join_split_string_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

spam = '''Dear Alice,
# OUT: How have you been? I am fine.
# OUT: There is a container in the fridge
# OUT: that is labeled "Milk Experiment".

# OUT: Please do not drink it.
# OUT: Sincerely,
# OUT: Bob'''
spam.split('\n')
# OUT: ['Dear Alice,', 'How have you been? I am fine.', 'There is a container in the
# OUT: fridge', 'that is labeled "Milk Experiment".', '', 'Please do not drink it.',
# OUT: 'Sincerely,', 'Bob']
```

Topic: Justifying Text with `rjust()`, `ljust()`, and `center()`

26/46 26_justifying_text_rjust.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> 'Hello'.rjust(10)  
# OUT: '      Hello'  
>>> 'Hello'.rjust(20)  
# OUT: '           Hello'  
>>> 'Hello World'.rjust(20)  
# OUT: '        Hello World'  
>>> 'Hello'.ljust(10)  
# OUT: 'Hello      '
```

27/46 27_justifying_text_rjust_hello.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> 'Hello'.rjust(20, '*')  
# OUT: '*****Hello'  
  
>>> 'Hello'.ljust(20, '-')  
# OUT: 'Hello-----'
```

28/46 28_justifying_text_rjust_hello_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> 'Hello'.center(20)  
# OUT: '          Hello          '  
>>> 'Hello'.center(20, '=')  
# OUT: '=====Hello====='
```

29/46 29_justifying_text_rjust_printpicnic.py [script]

Script: save and run; predict the output first, then compare.

```
def printPicnic(itemsDict, leftWidth, rightWidth):
    print('PICNIC ITEMS'.center(leftWidth + rightWidth, '-'))
    for k, v in itemsDict.items():
        print(k.ljust(leftWidth, '.') + str(v).rjust(rightWidth))
picnicItems = {'sandwiches': 4, 'apples': 12, 'cups': 4, 'cookies': 8000}
printPicnic(picnicItems, 12, 5)
printPicnic(picnicItems, 20, 6)
```

30/46 30_justifying_text_rjust_picnic.py [script]

Script: save and run; predict the output first, then compare.

```
---PICNIC ITEMS--
sandwiches..    4
apples.....   12
cups.....      4
cookies..... 8000
-----PICNIC ITEMS-----
sandwiches.....    4
apples.....       12
cups.....          4
cookies.....     8000
```

Topic: Removing Whitespace with strip(), rstrip(), and lstrip()

31/46 31_removing_whitespace_strip.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> spam = '    Hello World    '  
>>> spam.strip()  
# OUT: 'Hello World'  
>>> spam.lstrip()  
# OUT: 'Hello World '  
>>> spam.rstrip()  
# OUT: '    Hello World'
```

32/46 32_removing_whitespace_strip_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = 'SpamSpamBaconSpamEggsSpamSpam'  
spam.strip('ampS')  
# OUT: 'BaconSpamEggs'
```

Topic: Copying and Pasting Strings with the pyperclip Module

33/46 33_copying_pasting_strings.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pyperclip
pyperclip.copy('Hello world!')
pyperclip.paste()
# OUT: 'Hello world!'
```

34/46 34_copying_pasting_strings_pyperclip.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
pyperclip.paste()  
# OUT: 'For example, if I copied this sentence to the clipboard and then called  
# OUT: paste(), it would look like this:'
```

Topic: Step 1: Program Design and Data Structures

35/46 35_step_1_program.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# pw.py - An insecure password locker program.

PASSWORDS = {'email': 'F7minlBDDuvMJuxESSKHFhTxFtjVB6',
              'blog': 'VmALvQyKAxiVH5G8v01if1MLZF3sdt',
              'luggage': '12345'}
```

Topic: Step 2: Handle Command Line Arguments

36/46 36_step_2_handle.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# pw.py - An insecure password locker program.

PASSWORDS = {'email': 'F7minlBDDuvMJuxESSKHFhTxFtjVB6',
              'blog': 'VmALvQyKAxiVH5G8v01if1MLZF3sdt',
              'luggage': '12345'}

import sys
if len(sys.argv) < 2:
    print('Usage: python pw.py [account] - copy account password')
    sys.exit()

account = sys.argv[1]      # first command line arg is the account name
```

Topic: Step 3: Copy the Right Password

37/46 37_step_3_copy.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# pw.py - An insecure password locker program.
PASSWORDS = {'email': 'F7minlBDDuvMJuxESSKHFhTxFtjVB6',
              'blog': 'VmALvQyKAxiVH5G8v01if1MLZF3sdt',
              'luggage': '12345'}

import sys, pyperclip
if len(sys.argv) < 2:
    print('Usage: py pw.py [account] - copy account password')
    sys.exit()

account = sys.argv[1] # first command line arg is the account name

if account in PASSWORDS:
    pyperclip.copy(PASSWORDS[account])
    print('Password for ' + account + ' copied to clipboard.')
else:
    print('There is no account named ' + account)
```

38/46 38_step_3_copy_2.py [script]

Script: save and run; predict the output first, then compare.

```
@py.exe C:\Python34\pw.py %*  
@pause
```

Topic: Project: Adding Bullets to Wiki Markup

39/46 39_project_adding_bullets.py [script]

Script: save and run; predict the output first, then compare.

Lists of animals

Lists of aquarium life

Lists of biologists by author abbreviation

Lists of cultivars

40/46 40_project_adding_bullets_lists.py [script]

Script: save and run; predict the output first, then compare.

```
* Lists of animals
* Lists of aquarium life
* Lists of biologists by author abbreviation
* Lists of cultivars
```

Topic: Step 1: Copy and Paste from the Clipboard

41/46 41_step_1_copy.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# bulletPointAdder.py - Adds Wikipedia bullet points to the start
# of each line of text on the clipboard.

import pyperclip
text = pyperclip.paste()

# TODO: Separate lines and add stars.

pyperclip.copy(text)
```

Topic: Step 2: Separate the Lines of Text and Add the Star

42/46 42_step_2_separate.py [script]

Script: save and run; predict the output first, then compare.

```
'Lists of animals\nLists of aquarium life\nLists of biologists by author  
abbreviation\nLists of cultivars'
```

43/46 43_step_2_separate_pyperclip.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# bulletPointAdder.py - Adds Wikipedia bullet points to the start
# of each line of text on the clipboard.

import pyperclip
text = pyperclip.paste()

# Separate lines and add stars.
lines = text.split('\n')
for i in range(len(lines)):    # loop through all indexes in the "lines" list
    lines[i] = '* ' + lines[i] # add star to each string in "lines" list

pyperclip.copy(text)
```

Topic: Step 3: Join the Modified Lines

44/46 44_step_3_join.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# bulletPointAdder.py - Adds Wikipedia bullet points to the start
# of each line of text on the clipboard.

import pyperclip
text = pyperclip.paste()

# Separate lines and add stars.
lines = text.split('\n')
for i in range(len(lines)):    # loop through all indexes for "lines" list
    lines[i] = '* ' + lines[i] # add star to each string in "lines" list
text = '\n'.join(lines)
pyperclip.copy(text)
```

Topic: Table Printer

45/46 45_table_printer.py [script]

Script: save and run; predict the output first, then compare.

```
tableData = [['apples', 'oranges', 'cherries', 'banana'],  
             ['Alice', 'Bob', 'Carol', 'David'],  
             ['dogs', 'cats', 'moose', 'goose']]
```

46/46 46_table_printer_apples.py [script]

Script: save and run; predict the output first, then compare.

```
apples Alice dogs
oranges Bob cats
cherries Carol moose
banana David goose
```